

# ATS & AI Voice Interview System Design

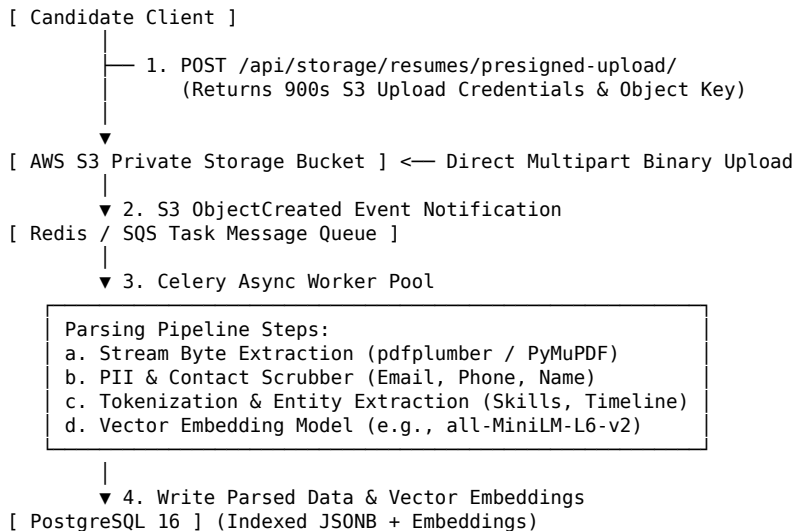
Zecpath AI Recruitment Platform

|                    |                                                                                                                         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Topics</b>      | ATS System Design, AI Voice Interview System Design, Scalability Considerations, End-to-End System & Data Flow Diagrams |
| <b>Prepared by</b> | <b>RAJKUMAR PR</b>                                                                                                      |

# 1. ATS System Design (Applicant Tracking System)

## 1.1 Resume Parsing Pipeline

Handling thousands of resumes daily requires an asynchronous, decoupled architecture to prevent multi-part file uploads and heavy PDF text extractions from tying up WSGI HTTP workers.



- **Direct-to-S3 Upload:** Web application instances never handle large binary payloads directly. Clients receive a pre-signed S3 URL and upload files directly to cloud storage, keeping web server bandwidth near zero for uploads.
- **Asynchronous Decoupling:** File processing is deferred to background worker processes via Redis task queues. If parsing spikes, the HTTP API remains responsive and returns immediately.

## 1.2 Candidate Ranking System

Candidate suitability is evaluated via high-dimensional vector math combined with rule-based criteria:

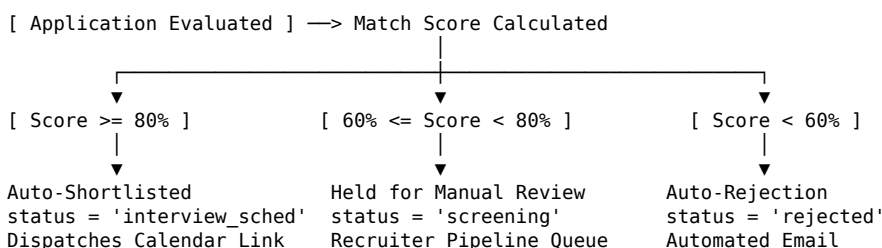
- **Semantic Match:** Cosine similarity is computed between the candidate's normalized resume embedding vector **A** and the job requirement vector **B**:

$$\text{Similarity} = \cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

- **Hard Filtering Gates:**
  - Minimum years of experience threshold.
  - Required tech stack tags (e.g., Python, PostgreSQL, Celery).
  - Location / Visa parameters.
- **Composite Normalization:** The match score is converted to a 0-100 scale and written to `core_application.match_score`, allowing recruiters to run instant  $O(\log N)$  sorted index queries using the composite index `idx_app_job_status`.

## 1.3 Shortlisting Automation

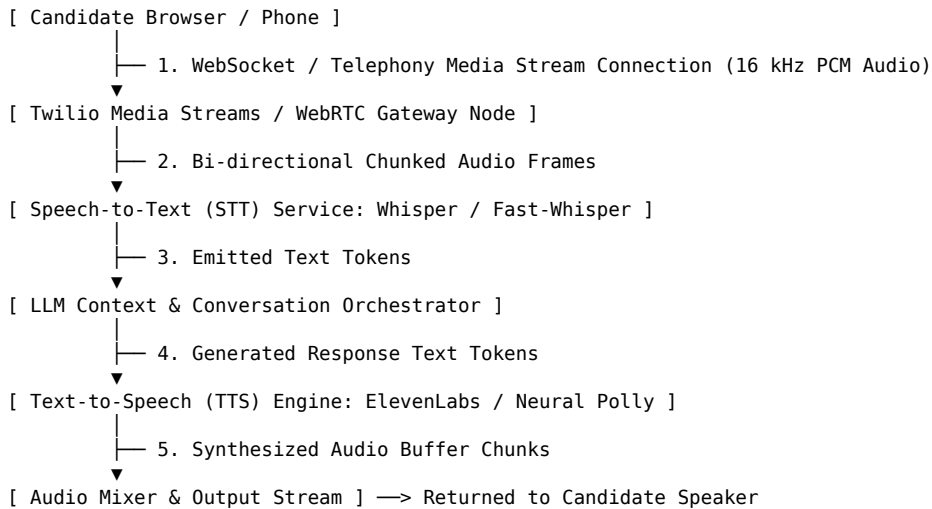
Automated shortlisting transitions candidate applications through the hiring funnel without manual intervention:



## 2. AI Voice Interview System Design

### 2.1 AI Call Flow

Voice interactions require a full-duplex, low-latency streaming pipeline to keep conversational delay under 300 ms:



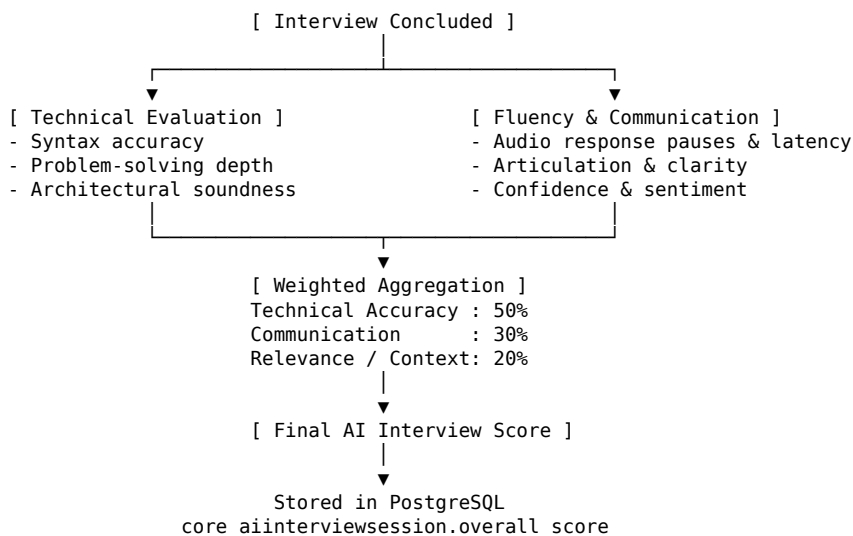
### 2.2 Question Engine

Rather than reading static scripts, the Question Engine generates adaptive follow-ups based on the candidate's actual answers:

- **Context Window Assembly:**
  - **System Prompt:** Defines role, strictness, and interview boundaries.
  - **Candidate Profile:** Extracted resume text and verified skills.
  - **Job Description:** Specific competencies to assess.
  - **Transcript History:** Prior question-and-answer exchanges in the active session.
- **Dynamic Generation:**
  - If the candidate gives a shallow answer, the engine drills deeper: *"You mentioned using Redis for caching; how did you handle cache stampedes?"*
  - If the candidate answers thoroughly, the engine advances to the next architectural requirement.

### 2.3 Scoring System

At the conclusion of the interview, the session audio and transcripts are passed to the automated scoring pipeline:



### 3. Scalability Considerations

#### 3.1 Queue-Based Processing (Decoupling)

To maintain high availability during traffic spikes, long-running operations are offloaded into dedicated, prioritized Redis queues:

| Queue Name     | Priority | Concurrency            | Target Task Duration | Workers Assigned |
|----------------|----------|------------------------|----------------------|------------------|
| voice_realtime | High     | Event-driven / AsyncIO | < 200 ms             | 8 Workers        |
| ats_parsing    | Medium   | Prefork (CPU-bound)    | 1-3 seconds          | 4 Workers        |
| notifications  | Low      | Gevent (I/O-bound)     | Network-bound        | 2 Workers        |

- **Backpressure Protection:** When the `ats_parsing` queue depth exceeds 1,000 tasks, auto-scaling spin-ups launch additional ephemeral worker instances.
- **Dead-Letter Queues (DLQ):** Tasks failing more than 3 times with unhandled exceptions are routed to a DLQ for manual inspection without stalling active queues.

#### 3.2 Architectural Pattern: Modular Monolith vs. Microservices

For a platform at this scale, choosing the right service boundaries prevents unnecessary operational complexity:

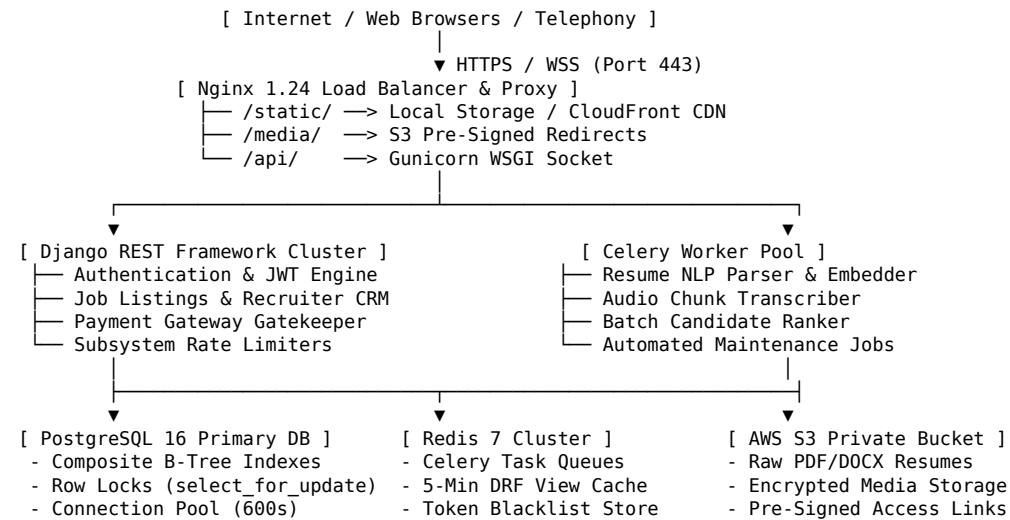
Tradeoff Comparison Matrix

| Dimension        | Modular Monolith (Chosen)     | Microservices                |
|------------------|-------------------------------|------------------------------|
| Deployment Ops   | Single CI/CD Pipeline         | Multi-cluster Kubernetes     |
| Network Latency  | Zero (In-memory calls)        | High (gRPC / HTTP hops)      |
| Data Consistency | ACID (PostgreSQL atomic)      | Eventual (2PC / Sagas)       |
| Operational Cost | Low (Single server / cluster) | High (Service mesh, tracing) |
| Scaling Profile  | Worker-level scaling          | Fine-grained pod scaling     |

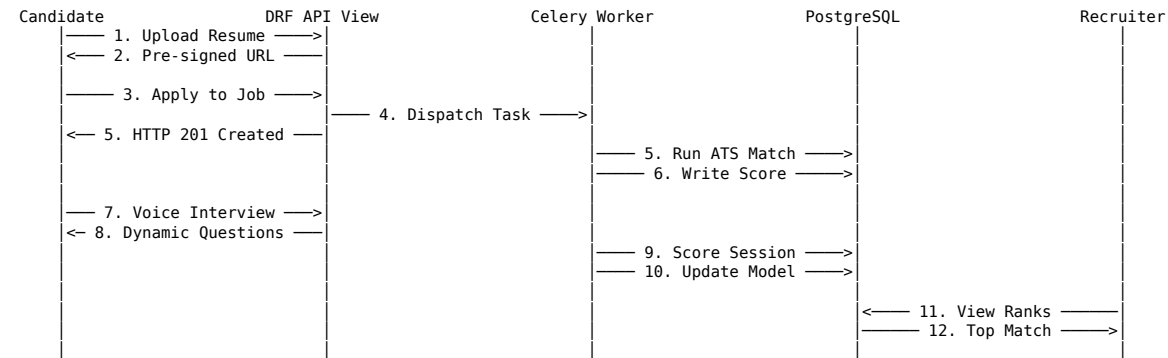
**The Zecpath Architecture Choice:** A Modular Monolith built on Django REST Framework and Celery. Core domains (auth, billing, ats, voice) are divided into isolated Python packages. Heavy lifting (parsing and voice streaming) scales horizontally as dedicated worker pools without introducing the operational overhead of a distributed microservices architecture.

## 4. End-to-End System & Data Flow Diagrams

### 4.1 Global System Architecture Diagram



### 4.2 Data Flow Diagram: Candidate Application to Offer



Prepared by **RAJKUMAR PR**